RTX Trading — Mobile API & WebSocket Integration Guide

> ***For mobile app developers*** *— everything you need to connect your app to the RTX trading backend.*

Table of Contents

1. System Architecture
2. Authentication Flow
3. REST API Reference
4. WebSocket Integration
5. All WebSocket Message Types
6. Code Examples
7. Error Handling & Reconnection

---

1. System Architecture

The backend consists of **three separate servers**. Mobile apps talk mostly to the **Gateway** (port 7999) and the **WebSocket** endpoint.

```
MOBILE APP
    │
    ├── REST ──────► Gateway Backend    (port 7999)  ← your main entry
    │                    │
    │                    ├── routes to ──► MT5 Sync C++ (port 8088) for trade execution
    │                    └── routes to ──► FIX API      (port 8081) for market data
    │
    └── WebSocket ──► wss://rtxbackend.tradecopier.org/fix-ws
                         (live ticks, candles, account, positions, orders)
```

Server	Base URL (Production)	Purpose
Gateway	`https://rtxbackend.tradecopier.org`	Auth, account, orders, positions
MT5 Sync (C++)	`http://localhost:8088` *(server-internal)*	Direct trade execution
WebSocket	`wss://rtxbackend.tradecopier.org/fix-ws`	All real-time streaming data

> *[!NOTE] As a mobile developer you only call two things: the* ***Gateway REST API*** *for data and actions, and the* ***WebSocket*** *endpoint for live streaming. You never talk to the MT5 Sync server directly — the Gateway proxies it for you.*

---

2. Authentication Flow

Step 1 — Login to get your token

```
POST /api/login
```

Request:

```
{
  "username": "900767",
  "password": "your_password"
}
```

Response:

```
{
  "access_token": "eyJhbGci...",
  "refresh_token": "eyJhbGci...",
  "expires_at": "2026-03-14T20:00:00Z",
  "user": {
    "id": "123",
    "username": "900767",
    "email": "trader@example.com",
    "role": "trader"
  }
}
```

Step 2 — Use the token in every request

All protected endpoints need this HTTP header:

```
Authorization: Bearer <access_token>
```

Step 3 — Use the token for WebSocket too

Append `?token=<access_token>` to the WebSocket URL:

```
wss://rtxbackend.tradecopier.org/fix-ws?token=eyJhbGci...
```

Step 4 — Refresh before expiry

```
POST /api/refresh
Body: { "refresh_token": "<your_refresh_token>" }
```

Token expiry handling

If the WebSocket closes with **code 1008** or **401**, the token has expired. Clear it, redirect the user to login, and repeat from Step 1.

---

3. REST API Reference

Base URL: `https://rtxbackend.tradecopier.org`

🔓 Public Endpoints (no token needed)

Health Check

```
GET /health
```

Returns: `{ "status": "healthy", "timestamp": 1741911600, "version": "1.0.0" }`

Login

```
POST /api/login
Body: { "username": "string", "password": "string" }
```

Logout

```
POST /api/logout
```

Refresh Token

```
POST /api/refresh
Body: { "refresh_token": "string" }
```

Register New MT5 Account

```
POST /api/mt5/account/create
```

Request:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+919876543210",
  "country": "India",
  "city": "Mumbai",
  "group": "demo\\STD",
  "leverage": 100
}
```

Response:

```
{
  "success": true,
  "message": "Account created",
  "account": {
    "login": 900789,
    "master_password": "Ux@O@w03Q7N@",
    "investor_password": "...",
    "group": "demo\\STD",
    "server": "...",
    "created_at": "2026-03-14T10:00:00Z"
  }
}
```

🔐 Protected Endpoints (Bearer token required)

MT5 Connection Status

```
GET /api/mt5/status
```

Returns: `{ "connected": true, "address": "...", "login": 900767, "last_error": "" }`

Get Account Groups

```
GET /api/mt5/groups
```

Returns: `{ "count": 5, "groups": [{ "name": "demo\\STD", "currency": "USD" }] }`

📊 Market Data

Get All Symbols

```
GET /api/symbols
Authorization: Bearer <token>
```

Get Live Quote

```
GET /api/quotes/{symbol}
Authorization: Bearer <token>
```

Example: `GET /api/quotes/EURUSD`

Returns: `{ "bid": 1.08512, "ask": 1.08525, "spread": 0.00013 }`

Get Historical Candles (OHLCV)

```
GET /api/candles/{symbol}?timeframe=H1&count=100
Authorization: Bearer <token>
```

Parameter	Type	Values	Default
`timeframe`	string	`M1`, `M5`, `M15`, `M30`, `H1`, `H4`, `D1`, `W1`, `MN`	`M1`
`count`	integer	max candles to return	100

Get Tick Data

```
GET /api/ticks/{symbol}
Authorization: Bearer <token>
```

💰 Account Info

Account Summary

```
GET /api/account/summary
Authorization: Bearer <token>
```

Response:

```
{
  "id": "acc_123",
  "balance": 10300.25,
  "equity": 10450.00,
  "margin_used": 150.00,
  "margin_available": 10150.00,
  "unrealized_pl": 149.75,
  "realized_pl": 0.50,
  "open_positions": 2,
  "pending_orders": 1,
  "currency": "USD",
  "leverage": 100
}
```

Transaction History

```
GET /api/account/transactions?type=trade&limit=50
Authorization: Bearer <token>
```

Query Param	Values
`type`	`deposit`, `withdrawal`, `trade`, `fee`, `commission`
`limit`	integer (default 50)

Equity History (for chart)

```
GET /api/account/equity-history?period=30d&interval=1d
Authorization: Bearer <token>
```

Param	Values
`period`	`1d`, `7d`, `30d`, `90d`, `1y`
`interval`	e.g. `1d`, `1h`

📋 Orders

Get All Orders

```
GET /api/orders
Authorization: Bearer <token>
```

Response:

```
{
  "orders": [
    {
      "id": "ord_abc",
      "symbol": "EURUSD",
      "type": "market",
      "side": "buy",
      "quantity": 0.01,
      "price": 1.08512,
      "status": "filled",
      "created_at": "2026-03-14T10:00:00Z",
      "mt5_order_id": 12345,
      "mt5_deal_id": 67890
    }
  ],
  "count": 1
}
```


Place Market Order

```
POST /api/orders
Authorization: Bearer <token>
```

Request:


```
{
  "symbol": "EURUSD",
  "type": "market",
  "side": "buy",
  "quantity": 0.01,
  "price": 0,
  "account_login": 900767
}
```


> *[!IMPORTANT] Only `"type": "market"` is currently supported. For **buy** set `"side": "buy"`, for **sell** set `"side": "sell"`.*

Response (201):


```
{
  "id": "ord_xyz",
  "symbol": "EURUSD",
  "type": "market",
  "side": "buy",
  "quantity": 0.01,
  "price": 1.08512,
  "status": "filled",
  "mt5_order_id": 12346,
```

```
  "mt5_deal_id": 67891
}
```

Cancel Order

```
DELETE /api/orders/{id}
Authorization: Bearer <token>
```

📈 Positions

Get Open Positions

```
GET /api/positions
Authorization: Bearer <token>
```

Response:

```
{
  "positions": [
    {
      "id": "pos_abc",
      "symbol": "EURUSD",
      "side": "long",
      "quantity": 0.01,
      "open_price": 1.08512,
      "current_price": 1.08600,
      "unrealized_pl": 8.80,
      "opened_at": "2026-03-14T10:00:00Z"
    }
  ],
  "count": 1
}
```

Close Position

```
POST /api/positions/{id}/close
Authorization: Bearer <token>
```

---

4. WebSocket Integration

Connection URL

```
wss://rtxbackend.tradecopier.org/fix-ws?token=<access_token>
```

Connection Lifecycle

```
CONNECT ──► onopen ──► Subscribe to symbols & channels
                              │
```

```
                    Receive messages (tick, update, mt5_positions, etc.)
                          |
                    Send ping every 30s ──► backend responds with pong
                          |
                    onclose ──► reconnect with exponential backoff
```

Reconnection Strategy

Attempt	Wait before retry
1st	1 s
2nd	2 s
3rd	4 s
4th	8 s
…	× 2 each time
Max	30 s (cap)
Max attempts	10, then give up

---

5. All WebSocket Message Types

Messages YOU SEND to the server

Subscribe to market symbols (live ticks)

```
{
  "type": "subscribe",
  "symbols": ["EURUSD", "BTCUSD", "XAUUSD"]
}
```

Subscribe to a named channel

```
{
  "type": "subscribe",
  "channel": "positions.900767"
}
```

Unsubscribe from a channel

```
{
  "type": "unsubscribe",
  "channel": "positions.900767"
}
```

Keepalive ping (send every 30 seconds)

```
{
  "type": "ping",
  "timestamp": 1741911600000
}
```

Messages YOU RECEIVE from the server

`tick` — Live price update (high frequency, multiple per second)

```
{
  "type": "tick",
  "symbol": "EURUSD",
  "bid": 1.08512,
  "ask": 1.08525,
  "spread": 0.00013,
  "timestamp": 1741911600123,
  "high24h": 1.09100,
  "low24h": 1.08200,
  "dailyChange": 0.0012,
  "lp": "LP1",
  "digits": 5
}
```

> *[!TIP]* `dailyChange` *= today's price change from open.* `lp` *= liquidity provider name. Use* `bid` *for sell price,* `ask` *for buy price.*

`update` — Live candle update

```
{
  "type": "update",
  "symbol": "EURUSD",
  "candle": {
    "time": 1741911600,
    "open": 1.08490,
    "high": 1.08530,
    "low": 1.08480,
    "close": 1.08512,
    "volume": 423
  }
}
```

> *Updates the* ***current (last) candle*** *in real-time. Timeframe defaults to M1.*

`symbols` — Available trading instruments list

```
{
  "type": "symbols",
  "symbols": [
    {
```

```
      "symbol": "EURUSD",
      "displayName": "EUR/USD",
      "baseCurrency": "EUR",
      "quoteCurrency": "USD",
      "minLotSize": 0.01,
      "maxLotSize": 100.0,
      "lotStep": 0.01,
      "type": "forex"
    }
  ]
}
```

> *Sent automatically right after connecting. Use this to populate your symbol list.*

`mt5_positions` — Live open positions push

```
{
  "type": "mt5_positions",
  "positions": [
    {
      "position_id": 1234,
      "symbol": "EURUSD",
      "action": 0,
      "volume_lots": 0.01,
      "price_open": 1.08512,
      "price_current": 1.08600,
      "price_sl": 1.08200,
      "price_tp": 1.09000,
      "profit": 8.80,
      "storage": -0.12,
      "commission": -0.05,
      "time_create": 1741911600
    }
  ]
}
```

> *`action: 0` = BUY, `action: 1` = SELL*

`mt5_orders` — Live pending orders push

```
{
  "type": "mt5_orders",
  "orders": [
    {
      "ticket": 5001,
      "symbol": "XAUUSD",
      "type": 2,
      "volume_current": 0.10,
      "price_open": 2100.00,
      "price_current": 2098.50,
      "price_sl": 2080.00,
```

```
      "price_tp": 2150.00,
      "time_setup": 1741911000
    }
  ]
}
```

`mt5_account` — **Live account balance / equity update**

```
{
  "type": "mt5_account",
  "account": {
    "balance": 10300.25,
    "equity": 10450.00,
    "margin": 150.00,
    "margin_free": 10150.00,
    "profit": 149.75,
    "margin_level": 6966.67,
    "currency": "USD",
    "leverage": 100
  }
}
```

`channel_message` — **Positions/account via named channel**

```
{
  "type": "channel_message",
  "channel": "positions.900767",
  "data": {
    "positions": [ ... ]
  }
}
```

```
{
  "type": "channel_message",
  "channel": "account.900767",
  "data": {
    "account": {
      "balance": 10300.25,
      "equity": 10450.00,
      "margin": 150.00,
      "margin_free": 10150.00,
      "profit": 149.75,
      "margin_level": 6966.67,
      "currency": "USD",
      "leverage": 100
    }
  }
}
```

`pong` — Response to your `ping`

```
{ "type": "pong" }
```

Message Flow Summary

```
you ──► { type: "subscribe", symbols: ["EURUSD"] }

server ──► { type: "tick", symbol: "EURUSD", bid: ..., ask: ... }   ← every tick
server ──► { type: "update", symbol: "EURUSD", candle: { ... } }    ← every candle update
server ──► { type: "mt5_positions", positions: [ ... ] }            ← account changes
server ──► { type: "mt5_account", account: { ... } }                ← balance changes

you ──► { type: "ping", timestamp: 1741911600000 }                  ← every 30s
server ──► { type: "pong" }
```

---

6. Code Examples

Flutter/Dart — Full WebSocket Client

```dart
import 'dart:async';
import 'dart:convert';
import 'package:web_socket_channel/web_socket_channel.dart';

class TradingWebSocket {
  WebSocketChannel? _channel;
  Timer? _pingTimer;
  String _token;

  static const wsUrl = 'wss://rtxbackend.tradecopier.org/fix-ws';

  TradingWebSocket(this._token);

  void connect() {
    final uri = Uri.parse('$wsUrl?token=$_token');
    _channel = WebSocketChannel.connect(uri);

    _channel!.stream.listen(
      (message) => _onMessage(jsonDecode(message)),
      onError: (e) => print('WS Error: $e'),
      onDone: () {
        print('WS Closed');
        _reconnect();
      },
    );

    // Subscribe to symbols right after connecting
    _send({'type': 'subscribe', 'symbols': ['EURUSD', 'BTCUSD', 'XAUUSD']});
```

```
  // Start keepalive
  _pingTimer = Timer.periodic(Duration(seconds: 30), (_) {
    _send({'type': 'ping', 'timestamp': DateTime.now().millisecondsSinceEpoch});
  });
}

void _onMessage(Map<String, dynamic> msg) {
  switch (msg['type']) {
    case 'tick':
      final symbol = msg['symbol'];
      final bid = msg['bid'] as double;
      final ask = msg['ask'] as double;
      print('TICK $symbol: bid=$bid ask=$ask');
      // update your price state here
      break;

    case 'update':
      final candle = msg['candle'] as Map<String, dynamic>;
      print('CANDLE ${msg['symbol']}: close=${candle['close']}');
      // update your chart
      break;

    case 'symbols':
      final symbols = msg['symbols'] as List;
      print('Available symbols: ${symbols.length}');
      break;

    case 'mt5_positions':
      final positions = msg['positions'] as List;
      // update positions list
      break;

    case 'mt5_account':
      final account = msg['account'] as Map<String, dynamic>;
      final balance = account['balance'];
      final equity = account['equity'];
      print('Account: balance=$balance equity=$equity');
      break;

    case 'mt5_orders':
      final orders = msg['orders'] as List;
      // update orders list
      break;

    case 'pong':
      // connection alive
      break;
  }
}

void _send(Map<String, dynamic> message) {
```

```
    _channel?.sink.add(jsonEncode(message));
  }

  void _reconnect() {
    Future.delayed(Duration(seconds: 3), () => connect());
  }

  void dispose() {
    _pingTimer?.cancel();
    _channel?.sink.close();
  }
}
```

Flutter/Dart — REST API (Login + Place Order)

```
import 'dart:convert';
import 'package:http/http.dart' as http;

const baseUrl = 'https://rtxbackend.tradecopier.org';

// ── Login ──────────────────────────────────────────────────────
Future<String> login(String username, String password) async {
  final response = await http.post(
    Uri.parse('$baseUrl/api/login'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'username': username, 'password': password}),
  );
  final data = jsonDecode(response.body);
  return data['access_token']; // store this!
}

// ── Get Positions ──────────────────────────────────────────────
Future<List> getPositions(String token) async {
  final response = await http.get(
    Uri.parse('$baseUrl/api/positions'),
    headers: {'Authorization': 'Bearer $token'},
  );
  final data = jsonDecode(response.body);
  return data['positions'];
}

// ── Place Market Order ─────────────────────────────────────────
Future<Map> placeOrder(String token, {
  required String symbol,
  required String side,   // 'buy' or 'sell'
  required double quantity,
  required int accountLogin,
}) async {
  final response = await http.post(
    Uri.parse('$baseUrl/api/orders'),
    headers: {
```

```
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $token',
    },
    body: jsonEncode({
      'symbol': symbol,
      'type': 'market',
      'side': side,
      'quantity': quantity,
      'price': 0,
      'account_login': accountLogin,
    }),
  );
  return jsonDecode(response.body);
}

// ── Close Position ──────────────────────────────────────────
Future<void> closePosition(String token, String positionId) async {
  await http.post(
    Uri.parse('$baseUrl/api/positions/$positionId/close'),
    headers: {'Authorization': 'Bearer $token'},
  );
}
```

React Native / JavaScript

```
const BASE_URL = 'https://rtxbackend.tradecopier.org';
const WS_URL   = 'wss://rtxbackend.tradecopier.org/fix-ws';

// Login
async function login(username, password) {
  const res = await fetch(`${BASE_URL}/api/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password }),
  });
  const { access_token } = await res.json();
  return access_token;
}

// WebSocket connection
function connectWebSocket(token) {
  const ws = new WebSocket(`${WS_URL}?token=${token}`);

  ws.onopen = () => {
    // Subscribe to live prices
    ws.send(JSON.stringify({ type: 'subscribe', symbols: ['EURUSD', 'BTCUSD'] }));
  };

  ws.onmessage = ({ data }) => {
    const msg = JSON.parse(data);
    if (msg.type === 'tick') {
```

```
      console.log(`${msg.symbol}: bid=${msg.bid} ask=${msg.ask}`);
    } else if (msg.type === 'mt5_account') {
      console.log(`Balance: ${msg.account.balance}`);
    }
  };

  ws.onclose = () => setTimeout(() => connectWebSocket(token), 3000);
}
```

---

7. Error Handling & Reconnection

HTTP Status Codes

Code	Meaning	What to do
`200`	OK	Success
`201`	Created	Order placed
`400`	Bad Request	Check your request body
`401`	Unauthorized	Token expired → refresh or re-login
`404`	Not Found	Wrong ID
`502`	Bad Gateway	MT5 execution failed, retry
`500`	Server Error	Retry after a few seconds

WebSocket Close Codes

Code	Meaning	What to do
`1000`	Normal close	No action needed
`1008`	Policy violation (auth)	Token expired → re-login, reconnect
`401`	Unauthorized	Same as 1008
Any other	Network drop	Reconnect with exponential backoff

Key Rules for Mobile Developers

1. **Always store the token** in secure storage (Keychain on iOS, Keystore on Android).
2. **Subscribe to symbols right after** `onopen` fires — the initial `symbols` list arrives automatically.
3. **Ticks come very fast** (multiple per second per symbol). Throttle your UI updates.
4. **Never store candle data only from REST** — merge with live `update` WS events for real-time charts.
5. **On** `mt5_positions` — replace your entire positions list, don't try to merge.
6. **On** `mt5_account` — always update balance, equity, and margin atomically.
7. **Reconnect on any close** except code 1008/401 (those need re-login first).